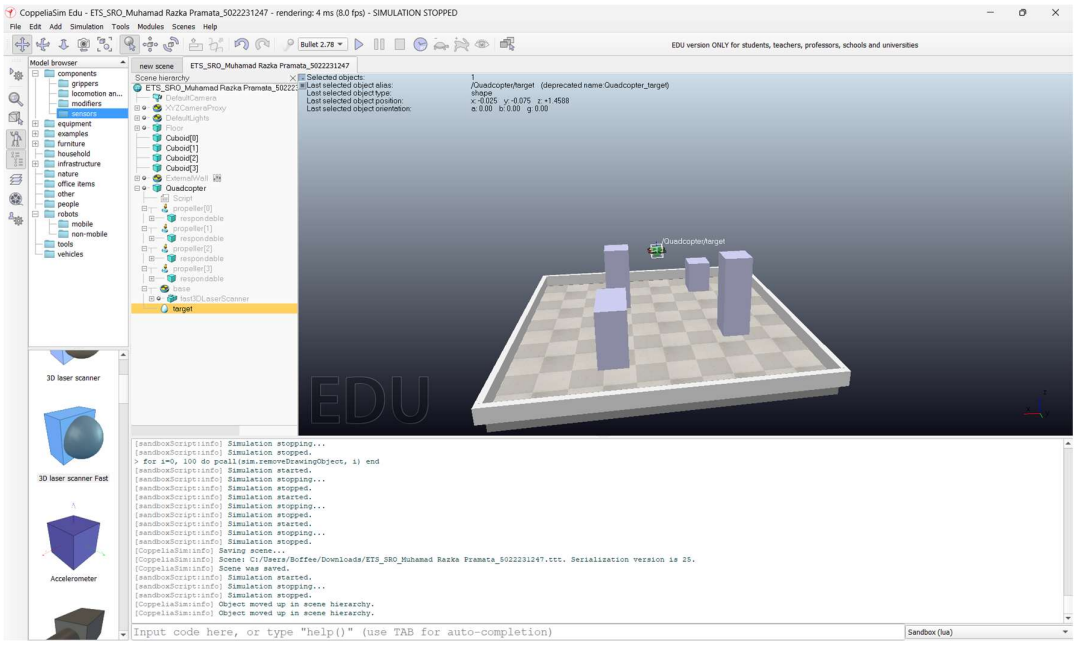
ETS SISTEM ROBOT OTONOM 3D MAPPING DENGAN DRONE



MUHAMAD RAZKA PRAMATA

5022231247

SCENE COPPELIASIM



Dalam CoppeliaSim saya membuat berupa area simple sebagai contoh yang akan dilakukan mapping dengan drone yang diberi sensor fast 3D laser scanner untuk melakukan 3D mapping. Drone tersebut akan digerakkan secara manual menggunakan mouse Ketika simulasi jalan dan akan mengscan apa yang di depan dan bawahny menggunakan fast 3D laser scannernya. Berikut script drone tersebut:

```
sim=require'sim'

function sysCall_init()
    particlesAreVisible=true
    simulateParticles=true
    fakeShadow=true

    particleCountPerSecond=430
    particleSize=0.005
    particleDensity=8500
    particleScatteringAngle=30
    particleLifeTime=0.5
    maxParticleCount=50

    -- Detatch the manipulation sphere:
    targetObj=sim.getObject('../target')
    sim.setObjectParent(targetObj,-1,true)

    -- This control algo was quickly written and is dirty and not optimal. It just serves as a SIMPLE example
    d=sim.getObject('../base')

    propellerHandles={}
    jointHandles={}
    particleObjects={-1,-1,-1,-1}
    local
ttype=sim.particle_roughspheres+sim.particle_cyclic+sim.particle_respondable1to4+sim.particle_respondable5to8+sim.part
icle_ignoresgravity
    if not particlesAreVisible then
        ttype=ttype+sim.particle_invisible
```

```
    end
    for i=1,4,1 do
        propellerHandles[i]=sim.getObject('../propeller['..(i-1)..']/respondable')
        jointHandles[i]=sim.getObject('../propeller['..(i-1)..']/joint')
        if simulateParticles then
            particleObjects[i]=sim.addParticleObject(ttype,particleSize,particleDensity,
{2,1,0.2,3,0.4},particleLifeTime,maxParticleCount,{0.3,0.7,1})
        end
    end
    heli=sim.getObject('..')

    pParam=2
    iParam=0
    dParam=0
    vParam=-2

    cumul=0
    lastE=0
    pAlphaE=0
    pBetaE=0
    psp2=0
    psp1=0

    prevEuler=0


    if (fakeShadow) then

 shadowCont=sim.addDrawingObject(sim.drawing_discpts+sim.drawing_cyclic+sim.drawing_25percenttransparency+sim.drawing_
50percenttransparency+sim.drawing_itemsizes,0.2,0,-1,1)
    end
end

function sysCall_cleanup()
    sim.removeDrawingObject(shadowCont)
    for i=1,#particleObjects,1 do
        sim.removeParticleObject(particleObjects[i])
    end
end

function sysCall_actuation()
    pos=sim.getObjectPosition(d)
    if (fakeShadow) then
        itemData={pos[1],pos[2],0.002,0,0,0,1,0.2}
        sim.addDrawingObjectItem(shadowCont,itemData)
    end

    -- Vertical control:
    targetPos=sim.getObjectPosition(targetObj)
    pos=sim.getObjectPosition(d)
    l=sim.getVelocity(heli)
    e=(targetPos[3]-pos[3])
    cumul=cumul+e
    pv=pParam*e
    thrust=5.45+pv+iParam*cumul+dParam*(e-lastE)+l[3]*vParam
    lastE=e

    -- Horizontal control:
    sp=sim.getObjectPosition(targetObj,d)
    m=sim.getObjectMatrix(d)
    vx={1,0,0}
    vx=sim.multiplyVector(m,vx)
    vy={0,1,0}
    vy=sim.multiplyVector(m,vy)
    alphaE=(vy[3]-m[12])
    alphaCorr=0.25*alphaE+2.1*(alphaE-pAlphaE)
    betaE=(vx[3]-m[12])
    betaCorr=-0.25*betaE-2.1*(betaE-pBetaE)
    pAlphaE=alphaE
    pBetaE=betaE
    alphaCorr=alphaCorr+sp[2]*0.005+1*(sp[2]-psp2)
    betaCorr=betaCorr-sp[1]*0.005-1*(sp[1]-psp1)
    psp2=sp[2]
    psp1=sp[1]

    -- Rotational control:
    euler=sim.getObjectOrientation(d,targetObj)
    rotCorr=euler[3]*0.1+2*(euler[3]-prevEuler)
    prevEuler=euler[3]

    -- Decide of the motor velocities:
    handlePropeller(1,thrust*(1-alphaCorr+betaCorr+rotCorr))
    handlePropeller(2,thrust*(1-alphaCorr-betaCorr-rotCorr))
    handlePropeller(3,thrust*(1+alphaCorr-betaCorr+rotCorr))
    handlePropeller(4,thrust*(1+alphaCorr+betaCorr-rotCorr))
end


function handlePropeller(index,particleVelocity)
    propellerRespondable=propellerHandles[index]
    propellerJoint=jointHandles[index]
    propeller=sim.getObjectParent(propellerRespondable)
    particleObject=particleObjects[index]
    maxParticleDeviation=math.tan(particleScatteringAngle*0.5*math.pi/180)*particleVelocity
    notFullParticles=0
```

```
    local t=sim.getSimulationTime()
    sim.setJointPosition(propellerJoint,t*10)
    ts=sim.getSimulationTimeStep()

    m=sim.getObjectMatrix(propeller)
    particleCnt=0
    pos={0,0,0}
    dir={0,0,1}

    requiredParticleCnt=particleCountPerSecond*ts+notFullParticles
    notFullParticles=requiredParticleCnt % 1
    requiredParticleCnt=math.floor(requiredParticleCnt)
    while (particleCnt<requiredParticleCnt) do
        -- we want a uniform distribution:
        x=(math.random()-0.5)*2
        y=(math.random()-0.5)*2
        if (x*x+y*y<=1) then
            if (simulateParticles) then
                pos[1]=x*0.08
                pos[2]=y*0.08
                pos[3]=-particleSize*0.6
                dir[1]=pos[1]+(math.random()-0.5)*maxParticleDeviation*2
                dir[2]=pos[2]+(math.random()-0.5)*maxParticleDeviation*2
                dir[3]=pos[3]-particleVelocity*(1+0.2*(math.random()-0.5))
                pos=sim.multiplyVector(m,pos)
                dir=sim.multiplyVector(m,dir)
                itemData={pos[1],pos[2],pos[3],dir[1],dir[2],dir[3]}
                sim.addParticleObjectItem(particleObject,itemData)
            end
            particleCnt=particleCnt+1
        end
    end
    -- Apply a reactive force onto the body:

 totalExertedForce=particleCnt*particleDensity*particleVelocity*math.pi*particleSize*particleSize*particleSize/(6*ts)
    force={0,0,totalExertedForce}
    m[4]=0
    m[8]=0
    m[12]=0
    force=sim.multiplyVector(m,force)
    local rotDir=1-math.mod(index,2)*2
    torque={0,0,rotDir*0.002*particleVelocity}
    torque=sim.multiplyVector(m,torque)
    sim.addForceAndTorque(propellerRespondable,force,torque)
end
```

Untuk melakukan 3D mapping dan membuat 3D mapnya, saya menggunakan phyton dengan kode programnya sebagai berikut:

```
import math
import time
import numpy as np
import matplotlib.pyplot as plt
from coppeliasim_zmqremoteapi_client import RemoteAPIClient

print("Menghubungkan ke CoppeliaSim...")
client = RemoteAPIClient()
sim = client.require('sim')

# 1. Ambil handle objek dari CoppeliaSim
drone_base = sim.getObject('/Quadcopter')
vision_sensor = sim.getObject('/Quadcopter/fast3DLaserScanner/sensor')

# Ambil parameter kamera depth
fov = sim.getObjectFloatParam(vision_sensor, sim.visionfloatparam_perspective_angle)
max_dist = 3.5  # Batas jarak maksimum deteksi (meter)

# Buat objek gambar garis biru (jalur) & titik merah (sensor) di CoppeliaSim
line_container = sim.addDrawingObject(sim.drawing_lines, 3, 0, -1, 99999, [0, 0, 1])
point_container = sim.addDrawingObject(sim.drawing_points, 4, 0, -1, 10000, [1, 0, 0])

# ... (kode bagian atas tetap sama) ...

trajectory_data = []
point_cloud_data = []
last_pos = None
```

```python
# 1. NYALAKAN SIMULASI DULU
sim.startSimulation()
time.sleep(0.2)

# 2. BARU BUAT DRAWING OBJECT (supaya ikut ter-reset bersama simulasi)
line_container = sim.addDrawingObject(sim.drawing_lines, 3, 0, -1, 99999, [0, 0, 1])
point_container = sim.addDrawingObject(sim.drawing_points, 4, 0, -1, 10000, [1, 0, 0])

print("Simulasi berjalan! Silakan geser bola hijau 'target' di CoppeliaSim dengan mouse.")
print("Tekan Ctrl+C di terminal VS Code (atau tombol Stop di CoppeliaSim) jika sudah selesai keliling.")

try:
    while sim.getSimulationState() != sim.simulation_stopped:
        # --- A. Rekam Posisi Drone (Grade 1 - C) ---
        pos = sim.getObjectPosition(drone_base, -1)
        trajectory_data.append(pos)

        if last_pos is not None:
            sim.addDrawingObjectItem(line_container, last_pos + pos)
        last_pos = pos

        # --- B. Baca Sensor & Konversi ke 3D Point Cloud (Grade 2 & 3) ---
        # Langsung ambil depth buffer (tanpa handleVisionSensor karena bukan explicit handling)
        depth_raw, res = sim.getVisionSensorDepth(vision_sensor, 1)
        depth_buffer = sim.unpackFloatTable(depth_raw)
        sensor_matrix = sim.getObjectMatrix(vision_sensor, -1)

        if len(depth_buffer) > 0:
            # Ubah ke matriks 2D sesuai resolusi sensor (Y, X)
            depth_img = np.array(depth_buffer).reshape((res[1], res[0]))

            # Sampling grid (ambil sekitar 24x24 titik supaya ringan & rapi seperti video dosen)
            step_x = max(1, res[0] // 24)
            step_y = max(1, res[1] // 24)

            y_idx, x_idx = np.mgrid[0:res[1]:step_y, 0:res[0]:step_x]
            d = depth_img[y_idx, x_idx]

            # Filter hanya titik yang mengenai objek/lantai (0.05m < jarak < max_dist)
            valid = (d > 0.05) & (d < max_dist)

            if np.any(valid):
                d_valid = d[valid]
                x_valid = x_idx[valid]
                y_valid = y_idx[valid]

                norm_x = ((x_valid + 0.5) / res[0]) - 0.5
                norm_y = 0.5 - ((y_valid + 0.5) / res[1])
                tan_half_fov = math.tan(fov / 2.0)

                vx = -norm_x * 2.0 * tan_half_fov * d_valid
                vy = norm_y * 2.0 * tan_half_fov * d_valid
                vz = d_valid

                # Transformasi ke World Coordinate sekaligus dengan matriks NumPy
                M = np.array(sensor_matrix).reshape(3, 4)
                R = M[:, :3]
                t = M[:, 3]

                local_pts = np.vstack((vx, vy, vz))  # Shape: (3, N)
                world_pts = (R @ local_pts).T + t    # Shape: (N, 3)

                point_cloud_data.append(world_pts)

                # Gambar sampel titik merah di CoppeliaSim (dibatasi agar tidak lag ZMQ)
                sim.addDrawingObjectItem(point_container, None)
                for pt in world_pts[::2]:
                    sim.addDrawingObjectItem(point_container, pt.tolist())

except KeyboardInterrupt:
    print("\nMenghentikan pengambilan data...")
except Exception as e:
    print(f"\nSimulasi berhenti: {e}")
finally:
    # Bersihkan garis biru dan titik merah dari layar sebelum stop
    try:
        sim.removeDrawingObject(line_container)
        sim.removeDrawingObject(point_container)
    except:
        pass
    sim.stopSimulation()
```

```python
# ==========================================================================
# POST-PROCESSING: Voxel Grid & Color Mapping Ketinggian (Grade 4 & 5)
# ==========================================================================
print("Memproses 3D Voxel Map...")
if len(point_cloud_data) == 0:
    print("Tidak ada titik yang terekam! Pastikan arah sensor menghadap ke bawah/objek.")
    exit()

traj = np.array(trajectory_data)
points = np.vstack(point_cloud_data)

# Simpan ke CSV di folder ETS SRO kamu
np.savetxt("drone_trajectory.csv", traj, delimiter=",", header="x,y,z", comments="")
np.savetxt("point_cloud.csv", points, delimiter=",", header="x,y,z", comments="")
print("Data mentah berhasil disimpan ke drone_trajectory.csv dan point_cloud.csv!")

# 1. Voxel Grid Downsampling (Grade 4 - AB)
voxel_size = 0.08  # Ukuran grid voxel 8 cm
voxel_indices = np.floor(points / voxel_size).astype(int)
unique_voxels = np.unique(voxel_indices, axis=0)
voxel_points = unique_voxels * voxel_size + (voxel_size / 2.0)

# Isi pilar titik vertikal ke bawah seperti contoh video kedua dosen
filled_voxels = []
for vx, vy, vz in voxel_points:
    if vz > 0.05:
        for z_val in np.arange(0.0, vz, voxel_size):
            filled_voxels.append([vx, vy, z_val])
    else:
        filled_voxels.append([vx, vy, vz])

filled_voxels = np.unique(np.round(filled_voxels, 4), axis=0)
print(f"Total titik mentah: {len(points)} -> Total Voxel setelah post-process: {len(filled_voxels)}")

# 2. Visualisasi 3D Color Mapping (Grade 5 - A)
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

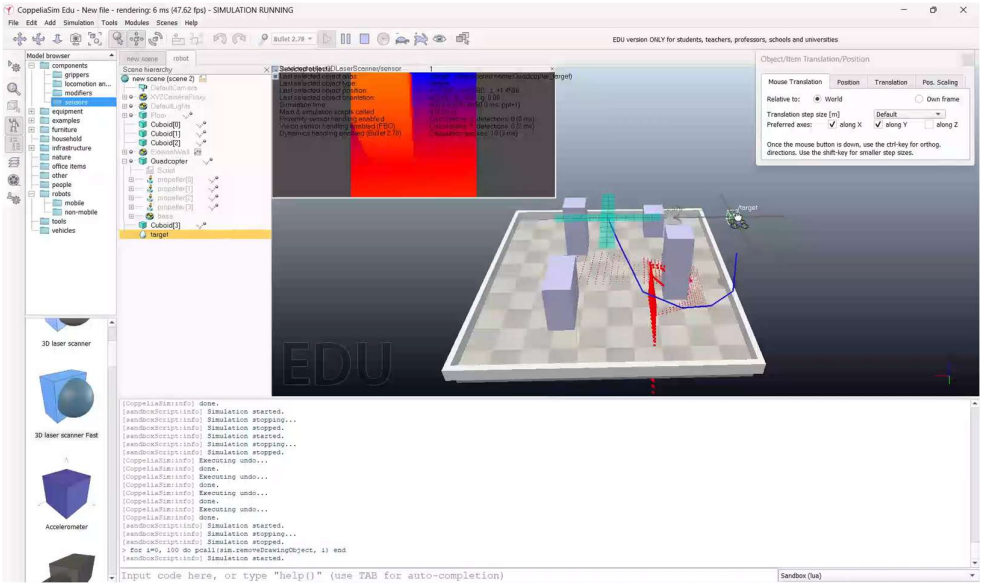
# Plot Jalur Drone (Biru)
ax.plot(traj[:, 0], traj[:, 1], traj[:, 2], color='blue', linewidth=2, label='Jalur Drone')

# Plot Voxel dengan warna berdasarkan ketinggian Z
scatter = ax.scatter(
    filled_voxels[:, 0],
    filled_voxels[:, 1],
    filled_voxels[:, 2],
    c=filled_voxels[:, 2],
    cmap='RdYlGn_r',
    s=8,
    alpha=0.85
)

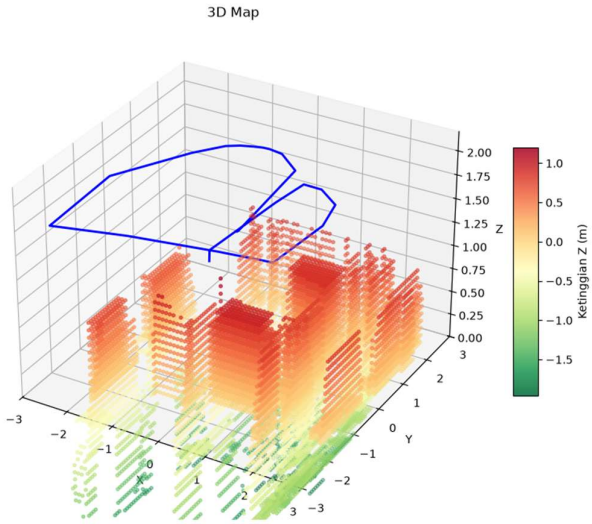
ax.set_title('3D Map')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_xlim([-3, 3])
ax.set_ylim([-3, 3])
ax.set_zlim([0, 2.2])

plt.colorbar(scatter, ax=ax, shrink=0.5, aspect=10, label='Ketinggian Z (m)')
plt.show()
```

Dengan bukti proses mapping adalah sebagai berikut:



Dengan hasil 3D mapping sebagai berikut:



Berikut link github yang berisi file .ttt, .py, dan .lua:

https://github.com/BoffeeSkelet/ETS_SRO_3Dmappingdengandrone_Muhamad-Razka-Pramata_5022231247

Berikut link Google Drive yang berisi video demo proses mapping dan 3D mapping:

https://drive.google.com/drive/folders/1gZSp1ZL31HeYYTmg0oMF8ebZbWjC0KQF?usp=drive_link